- **本体大部分学习由 ChatGPT 教授，另外还观看了 B 站、YouTube 上的相关视频**。
- B 站：跟李沐学 AI ———— 49.样式迁移（动手学神经网络v2）
- 与 ChatGPT 的对话链接：
  - https://chatgpt.com/share/6ad8c23b-e4e8-83e8-93e4-80e04ffb5e3a
  - https://chatgpt.com/share/6ad8c24d-6f2c-83ee-a1b7-08658fe37d4a
- **本学习笔记并非小登独立完成**。由于小登英语不好，写英语笔记的过程中会让 ChatGPT 修改、润色。

Cyber Painter

- **Image Style Transfer** aims to preserve the content of one image while adopting visual characteristics from another image.
- **If you want to produce a new image, the essence is to identify the pixels of the images.**

Model Pretraining and stunning

- **Feature Extraction**: The process of obtaining useful information from an image.
  - Different neural network layers may learn representations of edges, textures, shapes, and more complex visual patterns.
- **CNN (Convolutional Neural Network)**: A neural network designed to process data with spatial structure, such as images.
  - Convolution layers apply learnable numerical operations across local regions to extract useful patterns.
  - **Because it uses convolution to process the image, it can exploit spatial relationships in images. Its convolutional layers learn to respond to useful visual patterns.**
- **Parameter Initialization**: assign initial values to parameters.
- **Pretraining**: Training a model on an earlier dataset or task so that it acquires model structure and parameters that can be used later.
- **微调（Fine-tuning）**: Starting from pretrained model and continuing training on data from a new task so that the model adapts to that task.
  - In this process, we update some or all parameters of the pretrained model.
- **Advantages**:
  1. **Improved training efficiency and rate.**
  2. **Lower training cost and demand for equipment**。
  3. **Reduced dependence on large dataset.**
  4. **The new model inheritates the ability of pretrained model, so it may enhance the performance of the new model compared to training from scratch**。
- **PS: If the pretraining task and the new task share useful patterns, pretraining may improve efficiency and performance. However, if the tasks differ greatly or the pretrained parameters are unsuitable, the results may be poor. Therefore, experiments are needed to evaluate its effectiveness.**。
- **Fine-tuning continues updating model parameters after pretraining, but pretrained models can also be used without fine-tuning.
- **VGG**: refers to a family of convolutional neural networks developed by the Visual Geometry Group at the University of Oxford.

    - VGG19 doesn't contain Dropout in feature extraction layer, but in classification layers. VGG19 doesn't contain BatchNorm.
- **Model Family**: A group of neural networks that share similar design principles but differ in some architectural details.
- **ImageNet**: a widely used dataset in computer vision research that contains a numerous labeled images.
- **Why can style transfer use intermediate features from a pretrained VGG instead of only comparing corresponding pixels?**
    - Pixel-wise comparison alone is difficult to adequately separate image content from artistic style. Intermediate VGG representations provide learned visual information that can be used to construct content and style comparisons.
- **Optimization**: The process of repeatedly adjusting values that can be changed, according to a measure of how well the current result meets a given goal, so that the result gradually becomes closer to the desired outcome.
    - **Optimization Variable**: A value that we are allowed to change during optimization.
        - For instance, when training a neural network, the model parameters can be optimization variables. In classical neural style transfer, the pixel values of the generated image can be optimization variables.
- **Freezing Parameters**: Keeping selected model parameters unchanged during a particular training or optimization process.
    - **Main Purpose**:
        - **Preserving previously learned knowledge**. Useful features learned during pretraining are protected from unnecessary changes during subsequent training.
        - **Reducing training costs**. Freezing parameters reduces the number of parameter gradients that need to be computed and stored, as well as the number of parameters that must be updated. This can save computation and memory.
        - **Reducing the risk of overfitting**. Especially when the new dataset is small, freezing some parameters reduces the model's trainable degrees of freedom and may improve its ability to generalize to unseen data.
        - **Keeping feature extraction consistent**. In classical neural style transfer, freezing a pretrained VGG network allows it to use the same learned computations throughout optimization, providing a stable basis for comparing generated images.
    - **Common Strategy**:
        - Freeze the part transforms raw image data into useful numerical representations and train predicting part.
        - Freeze some earlier layers and update selected later layers.
        - Allow all relevant network layers to update their parameters.
- **Feature Representations**: Numerical descriptions of the image produced by the network's computations.
- **Content Loss** measures differences between the content feature representations of the generated image and the content image.
- **Style Loss** measures differences between the style feature representations of the generated image and the style image.
- **The Purpose of Pretrained Model in Style Transfer**: The pretrained model mainly serves as a feature extractor.
- **BatchNorm**:

    - In **training mode**, BatchNorm performs two different tasks:
        1. It uses the current batch's mean and variance to normalize the current data.
        2. It uses the current batch's statistics to update and store running statistics for future evaluation.
    - In **evaluation mode**, BatchNorm with its default settings no longer uses the current batch's statistics for normalization. Instead, it uses the running statistics stored during training.
- **There are actually three control steps in PyTorch:** Computation Mode, Gradient Computation, Parameter Update.

Import VGG

- The `torchvision.models` module provides many ready-made neural network architectures and their weights.
- `model = vgg19(weights=VGG19_Weights.DEFAULT)`
    - `vgg19()` is a function that creates a VGG19 network.
    - `weights=VGG19_Weights.DEFAULT` identifies the default pretrained weights provides by `torchvision` for VGG19.
- `model.features` accesses the feature-extraction component of VGG19.
    - This does not create a separate copy of the neural network. This return the existing feature-extraction component of VGG.
    - Special named attribute for VGG.
- In PyTorch, a trailing underscore `_` commonly indicates that an operation modifies the relevant state of the existing object rather than merely returning an independent new result.
- `torch.zeros()` creates a tensor in which every element is zero.
    - `test_image = torch.zeros((1, 3, 64, 64))` creates a four-dimensional tensor with the shape `(1, 3, 64, 64).
- `model.requires_grad_(False)` disables gradient computation for its parameters.
- **We still need to compute the gradient of the loss with respect to the generated image's pixels**. Therefore, we should not wrap the entire style transfer computation in `torch.no_grad()` merely to freeze VGG.
    - `torch.no_grad()` is a PyTorch tool that temporarily disables normal gradient recording. It is useful for operations that **do not require gradients**, but **wrapping the generated image's loss computation in it would prevent us from computing the gradients needed to update the image**.
- `.shape` is an attribute that reports the dimensions of a tensor.
- The output of any layers are **tensor**, not image.

Progressing a Real Image with VGG19

- **Resize**: Change an image's dimensions by enlarging or shrinking it. The program estimates new pixel values from the original pixels through a process called **interpolation**.
- **Center Crop**: Extract a region of a specified size from the center of an image. Some image content near the edges may be removed.
- `VGG19_Weights.DEFAULT.transforms()` create an image processing tool associated with those pretrained weights.
    - `transforms` is a method provided by Torchvision. It returns an object capable of processing images.

- `unsqueeze(dim)` inserts a new dimension of size 1 at the specified position in a tensor.
  - `dim` indicates the dimension, starting from 0.

Introduction to Style Transfer

- Why can't we simply average the corresponding pixel values of the sunflower photograph and The Starry Night and consider that a completed style transfer?
  - Because averaging pixel values merely blends the two images numerically. It does not distinguish which information represents the content to preserve and which represents the artistic style to imitate. The result may simply be a blended image rather than artwork with the desired content and style.
- **Feature Map**: a set of numerical values arranged spatially and produced by a layer of convolutional neural network.
- **Activation**: a numerical output produced by a neural unit.
  - A large activation does not necessarily mean the corresponding location is more important, nor does it necessarily represent an object that humans can clearly name.
- In a typical image classification CNN, **earlier layers** often represent **relatively simple local patterns, such as edges and color changes**. **Deeper layers** may represent more complex combinations of **shapes and visual structures.**
  - This is a common tendency, not a guarantee that every feature channel has a clear, unique interpretation.
- **Gram matrix**: A matrix formed from pairwise dot products between a collection of vectors.
  - **The Gram matrix can remain unchanged when spatial positions are rearranged in the same way across all feature channels**.
- **Content** is commonly measured by comparing **feature maps** from selected layers for the generated image and the content image. This comparison preserves information about the **spatial arrangement of features**.
- **Style** is commonly represented using **Gram matrices from multiple network layers**. These matrices describe **certain statistical relationships among feature channels**, allowing the generated image to imitate **texture and style patterns without strictly reproducing the original painting's spatial layout**.
- **Step**: Controls kernel movement and **affects feature-map information density**.
  - A smaller stride produces more output positions and denser spatial information, but requires more computation.
  - A larger stride produces fewer output positions and reduces computation, but may discard some details.
- **Content Loss**： A numerical measure of the difference between the generated image and the content image in selected neural network features.
  - Content loss usually requires corresponding feature representations to have **matching shapes**.
- **Style Loss**: A numerical measure of the difference in selected feature statistics between the generated image and the style image.
  - These statistics help us imitate certain textures and artistic characteristics of the style image.
  - In classical neural style transfer, **Gram matrices** are commonly used to describe overall dot-product relationships between feature channels.
  - Style loss based on Gram matrices can often compare **images with different spatial dimensions**.
    - **PS**: This based on Normalized Gram Matrix. For instance:

$$\widetilde{M}_{ij} = \frac{1}{HW} \sum_{p=1}^{HW} F_{i,p} F_{j,p}$$

- **Total Variation Loss**: When we include **total variation** as a loss to be reduced during optimization, it is called Total Variation Loss.
  - **Total Variation** is a mathematical quantity used to measure how much numerical values change between neighboring positions.
  - Total variation has several mathematical formulations. Some implementations use squared differences between neighboring pixels or divide the result by the number of pixels. Therefore, TV loss values from different implementations are not necessarily directly comparable.
- **Total Loss**: a numerical objective formed by combining multiple loss terms with selected weights.
  - The same two images can be preferred differently when we change the loss weights.
- **Convert tensor to image**:
  - **Denormalization / Unnormalization**: If the pixel values were previously standardized using means and standard deviations, they need to be converted back into their ordinary color-value range.
  - **Clamping**: Values outside an allowed interval are restricted to that interval.

Implementing the Program

Preprocessing

- `ImageOps.contain(image, size)`: It resizes an image so that it fits within the specified maximum dimensions while preserving its aspect ratio.
  - `ImageOps` is an image processing module provided by Pillow.
  - For instance: `ImageOps.contain(image, (256, 256))` fits the image within a maximum (256×256) area.
    - It does not force the output to be square. It only limits the output width and height to at most 256 pixels.
- `transforms.ToTensor()` converts **an ordinary Pillow RGB image into a PyTorch tensor**, scales typical 0–255 pixel-component values to floating-point values between **0 and 1** and changes the image representation from the common **(H, W, C) arrangement to (C, H, W)**.
- `transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])` standardizes the three color channels using specified means and standard deviations.
  - Why do mean and std each contain three numbers?
    - An RGB image has three color channels: Red, Green, and Blue. Each pixel contains one value for each channel. Each channel requires a mean and a std.

Typical Question and Program Implement

- **If the style image and content image have significantly different dimensions, will that affect the style transfer process?**
  - Yes. But a large difference in dimensions does not necessarily cause serious problems, nor must the two images have identical dimensions.
  - In classical neural style transfer, the generated image typically has the same dimensions as the content image to facilitate content-loss calculation. Style loss, however, is commonly calculated by comparing Gram matrices derived from VGG features. The dimensions of these matrices

depend on the number of feature channels rather than directly on image width and height. Therefore, the style image can have different spatial dimensions. Appropriate normalization can reduce numerical scale differences caused by different numbers of spatial positions.
    - Nevertheless, image dimensions still affect computational cost and visual quality. Higher-resolution images may preserve more details but require more computational resources. Resizing can also change texture scales or remove fine details, potentially influencing the final stylized result.
    - Therefore, the content and style images must have exactly the same dimensions. Instead, their resolutions should be selected based on computational resources, texture details, and the desired visual effect.

- **Briefly understand the mathematical principles of content loss, style loss, and total variation loss, and explain them in your own words**.

    - **Content Loss**:
        - Our goal is to transform the sunflower photograph into an oil painting while preserving its main structure.
        - If every RGB pixel had to remain identical to the original, the generated image would have little freedom to change its colors and textures.
        - Therefore, classical neural style transfer compares feature maps from an intermediate VGG19 layer rather than relying solely on raw pixel comparisons.
        - **Mean Squared Error(MSE)**:

$$L_{\text{content}} = \frac{1}{N} \sum_{k=1}^{N} \left(F_l(G)_k - F_l(C)_k \right)^2$$

            - C：The content image.
            - G：The generated image.
            - $F_l(I)$：he features of image I at selected VGG layer l.
            - N：the number of feature values being compared.
    - **Style Loss**:
        - A single dot product cannot prove proportionality or uniquely determine the similarity of two vectors.
        - Nevertheless, dot products **summarize aspects of how feature channels respond together across spatial positions**. Computing pairwise dot products among all channels produces a **Gram matrix**.

$$M_{ij}(F) = \frac{1}{HW} \sum_{p=1}^{HW} F_{i,p} F_{j,p}$$

            - i, j identify two feature channels.
            - p identifies a spatial position.
            - $F_{i,p}$ i The numerical value of channel i at p.
            - HW 特征通道的空间位置总量。
        - **MSE of Gram matrix**:

$$L_{\text{style}} = \frac{1}{C_f^2} \sum_{i=1}^{C_f} \sum_{j=1}^{C_f} \left(M_{ij}(G) - M_{ij}(S) \right)^2$$

        - $M(G)$ / $M(S)$: the Gram matrices of the generated and style images at the same selected layer.
        - C_f : The number of feature channels at the layer.
    - **Total Variation Loss**:
        - Content loss and style loss require features extracted by VGG19.
        - However, total variation loss can be computed directly from the generated image's pixels, without passing through VGG19.
        - It is commonly used as a **regularization term**.
        - Here, we want to **discourage excessive, unnecessary changes between neighboring pixels**.

$$L_{\text{TV}} = \frac{1}{(H-1)W} \sum_{i=1}^{H-1} \sum_{j=1}^{W} (P_{i+1,j} - P_{i,j})^2 + \frac{1}{H(W-1)} \sum_{i=1}^{H} \sum_{j=1}^{W-1} (P_{i,j+1} - P_{i,j})^2$$

            - $P_{i,j}$ The numerical value of a pixel at (i, j) in a channel.

- **What is the role of the Gram matrix in style loss? Why can dot products represent similarity between feature channels?**
    - **Role of Gram Matrix**:
        - A Gram matrix is primarily used to **describe statistical relationships between feature channels in a neural network**.
    - **Reasons**:
        - A dot product reflects **aspects of how two feature channels respond together across spatial positions**. When two channels tend to produce strong activations at corresponding positions, their dot product will often be larger. Therefore, the dot product provides one description of the relationship between channels.
        - Strictly speaking, however, a dot product is not equivalent to similarity. It depends on both **vector magnitudes and the angle** between them, so its value alone cannot determine whether two channels follow the same pattern.
        - The Gram matrix organizes pairwise channel dot products, **preserving certain activation-strength and co-activation statistics while discarding some exact spatial information**. This makes it useful for representing aspects of image texture and style. Classical neural style transfer constructs style loss by comparing the Gram matrices of the generated image and the style image.

- **If we consistently exchange spatial positions across all channels of an already extracted feature representation, its Gram matrix remains unchanged.**

- **Computational Graph**: A structure that represents dependencies between mathematical operations.
    - It describes which operations and variables contribute to the final output.
    - For instance:

- `x = torch.tensor(2.0, requires_grad=True)`

    - `requires_grad=True` tells PyTorch to track `relevant differentiable operations` so that gradients with respect to this tensor can be calculated.
        - The default is False.
    - `y = 3 * x` Because x requires gradients, the operation `y=3*x` is tracked.

- `y.detach()` returns a new tensor object containing the same values of y but disconnected from the earlier automatic differentiation graph.

    - The detached tensor normally **shares underlying data storage with the original tensor**, so detach() should **not** be treated as a guaranteed **independent data copy**.

- **Why Are the Target Features of the Content and Style Images Usually Detached from the Computational Graph?**

    - In classical neural style transfer, **we optimize the generated image's pixels, while the content and style images only provide fixed reference targets.**
    - Therefore, the target features extracted from the content and style images **do not need to participate in backpropagation**. Detaching these features can **avoid unnecessary gradient tracking, reduce associated computational and memory overhead, and explicitly treat them as fixed comparison targets**.
    - In PyTorch, we can use `.detach()` on already computed features or use `torch.no_grad()` when extracting reference features.
    - However, gradient tracking must remain enabled when processing the generated image through VGG, because backpropagation is needed to calculate the gradients of the loss with respect to its pixels.
    - Importantly, if VGG's parameters are already frozen and the reference images do not require gradients, the extracted target features normally have no backward-tracked computational graph. In that case, calling `.detach()` is not strictly necessary.

- **Forward Pass** computes **outputs and the loss from inputs**, following the ordinary direction of the operations.

- **Backward Pass, or Backpropagation**, starts from the final loss and applies differentiation rules to calculate **gradients** with respect to variables we want to optimize.

- `.clone()` creates a new tensor with copied data. The new tensor initially contains the same values, but it has independent underlying storage.

    - `.clone()` copies tensor data, but it **does not necessarily break gradient history**. If the original tensor requires gradients, its clone normally remains differentiably connected to it.
        - `a = b.clone()` give a connection like "a = b".

- `.item()` extracts a standard Python number from a tensor containing exactly one element.

- **Leaf Tensor** is a starting variable in the current automatic differentiation process, rather than an intermediate result produced by tracked operations.

    - After `.backward()` is called, PyTorch **stores gradients in the `.grad` property of leaf tensors that require gradients**.

    - **You can not do in-place operation to a leaf tensor**. If you do, the Python Interpreter will raise an error.
        - `RuntimeError`: a leaf Variable that requires grad is being used in an in-place operation.
        - Automatic differentiation must correctly track variables and their computational relationships, directly modifying a leaf tensor that requires gradients can interfere with the consistency of that tracking.

- `.grad_fn` accesses information about the operation used in automatic differentiation for that tensor.

    - For a non-leaf tensor produced by a gradient-tracked operation, grad_fn normally refers to the **corresponding backward-function object**.
    - **`grad_fn=None` does not mean that a tensor cannot receive gradients.**
        - If x is a starting variable. There is no preceding tracked differentiable operation that produced it.
        - Its gradient requirement is specified by requires_grad=True.
        - By contrast, y results from a tracked multiplication, so its grad_fn refers to the associated backward operation.

- **Buffers**: Running state values that need to be stored but are not learned using gradient descent. They were updated in training mode.

    - **Freezing model parameters does not automatically prevent updating buffers in training mode**.

- **`model.eval()` doesn't prevent Backward Pass.**

- **Is Setting a Model to Evaluation Mode the Same as Freezing Its Parameters During Transfer Learning?**

    - **NO**.
    - Evaluation mode controls **the behavior of certain network layers**.
    - Parameter freezing controls **whether model parameters require gradients**.

- To describe the architecture, we often group **several consecutive convolutional layers and their activation layers, followed by a pooling layer**, into a **convolutional block**.

- **Receptive Field**: The region of an input image that can influence a particular output position in a neural network layer.

- **How to choose the content layer and style layer?**

    - **Shallower** convolutional layers generally respond to local edges, color transitions, and simple textures. **Deeper** layers have larger receptive fields and can combine information from wider regions to form more complex visual features.
    - Therefore, content loss often uses a **relatively deep layer** to constrain the generated image's *__main structure while allowing changes in color and texture__.
    - Style loss commonly uses several convolutional layers at different depths. By comparing Gram matrices from these layers, we can **constrain texture and feature statistics at multiple spatial scales, producing a richer style representation**.

- In Python, a function itself is an object which can be assigned to a variable and passed to another function.

```
def say_hello():
    print("Hello")

say_hello()
```

```
def say_hello():
    print("Hello")

f = say_hello#there is no parenthesis are the name of the function.
f()
```

```
def calculate():
    print("Calculating...")
    return 10

def repeat_three_times(func):
    for i in range(3):
        result = func()
        print("Result:", result)

repeat_three_times(calculate)#there is no parenthesis are the name of
the function.
#If you add a parenthesis, the return result will save as the
parameter
```

- `.zero_grad()` clear the gredients of parameter atrribute, does not clear the historical information of L-BFGS.

- In L-BFGS:

  - The **search direction** is a vector specifying the direction of movement. However, it also has a magnitude.
  - The **step-length coefficient** is a scalar multiplying the search-direction vector.

- **When using L-BFGS, does one optimizer.step(closure) call necessarily evaluate the loss only once?**

  - One outer call can contain several internal iterations, and line search may require additional loss and gradient evaluations for different candidate step lengths.

- **Linear Search**: View the step-length coefficient as a varible. When we fixed current parament x_k and direction d_k, we try to find a proper step-length coefficient by substituding the step-length coefficient t into the Loss function and get a best t to minimize the Loss.

    - **Exact line search** finds the minimizer of $\phi(t)$. **Inexact line search** instead seeks a step satisfying suitable conditions without necessarily finding the exact minimum. This is often more practical for complicated neural-network losses.
        - PyTorch supports two settings: `None`, which skips the additional line-search procedure, and `"strong_wolfe"`, which searches for a step satisfying the Strong Wolfe conditions for parameter `line_search_fn=` in `torch.optim.LBFGS()`
            - The **Strong Wolfe conditions** check whether the **loss has decreased sufficiently and whether the directional derivative at the candidate point is sufficiently small in magnitude**.
                - **Sufficient Decrease**：

$$\phi(t) \leq \phi(0) + c_1 t \phi'(0)$$

                    - $\phi(0)$：Loss before moving.
                    - $\phi(t)$：Loss in the candidate position.
                    - c_1：A small positive number.
                - **Strong Curvature Condition**

$$\boxed{|\phi'(t)| \leq c_2 |\phi'(0)|}$$

                    - c_2 is a constant between 0 and 1. We ofen require $c_1 < c_2$.

- **The main loop consists of the following steps**:

    - **Clear previous gradients**: Clear gradients left from the previous computation to prevent unintended gradient accumulation.
    - **Extract features from the generated image**: Feed the current generated image into the fixed pretrained VGG19 network and extract feature maps from the required layers.
    - **Calculate the loss functions**: Compare the generated image's content features with the target features of the content image to calculate content loss. Compare correlations between feature channels of the generated image and the style image using Gram matrices to calculate style loss. Additionally, calculate total variation loss from differences between neighboring pixels.
    - **Calculate the total loss**: Compute a weighted sum of the three losses using hyperparameters.
    - **Perform backpropagation**: Calculate the gradient of the total loss with respect to the generated image's pixel values instead of updating the VGG19 parameters.
    - **Update the image using an optimizer**: Use an optimizer such as Adam, SGD, or L-BFGS to modify the generated image according to its gradients and the corresponding update rule.
    - **Repeat and record**: Repeat the process, record changes in the losses, and observe the generated results until the predefined iteration budget or stopping conditions are reached.

- `generated.clamp_(0.0, 1.0)` restrict values to a specified range.

    - In this example, the range is [0.0, 1.0]
    - The difference between `.clamp_()` and `.clamp()` is that clamp_(), whose name ends with an underscore, modifies the original tensor directly, while clamp() returns a new tensor.
    - `.clamp()` will make the gradient propagated back to the variable out of the range become zero.
        - **This does not mean that the loss has reached its minimum**. Rather, because the input is outside the clamping interval, the clamping function is locally insensitive to changes in

that input.
    - We uauslly use `with torch.no_grad(): x.clamp_()`. We generally do not want **the next backward pass to go through the previous clamping operation**, because that could cause **gradients to be reduced to zero** for inputs outside the permitted range.

- The **results of forward propagation** is/are **the loss/lossed** corresponding to the current generated image.
- The **results of backward propagate ** is the **gredient of the input tensor**.
    - **Backpropagation does not automatically update the generated image; the optimizer actually modifies its pixels**.
- `optimizer = torch.optim.SGD([x]/model.parameters(), lr=0.01)` The first parameter(`params`) tells the SGD optimizer which tensor to update.
    - **We use the list [x] instead of passing x directly**.
        - Because an optimizer may need to manage multiple tensors, the params argument expects an iterable, allowing the optimizer to obtain the tensors to be optimized one by one.
- A **hyperparameter** is a **configuration value** that we specify in advance or deliberately adjust, rather than a variable obtained directly through the current optimization process.
    - A **condition or setting chosen to control the behavior of a machine-learning model or optimization process**. It does **not have to be numerical**.
- **Gradient Norm** is the lenth of a gradient.
- Reasonable loss-weight tuning requires considering **loss values, gradient magnitudes, and the resulting images** together, rather than relying on a single indicator.
- **Loss weights** determine the relative importance of different objectives in neural style transfer. They should be selected experimentally using **loss definitions, numerical scales, gradients, and actual generated results**, rather than mechanically copying weight values from another implementation.
- `enumerate()` is a built-in Python function that provides **both the index and the element itself while iterating over a collection**.
- In `network = vgg19(weights=VGG19_Weights.DEFAULT).features.eval()`, `.eval()` directly set the model into evaluation model.
- When saving an image, `PIL` generally needs image data stored in **CPU** memory.
- When tensors are cloned, the new tensor will be kept in the same device with the old one